

TaskFlow

Arhitektonska dokumentacija

WPF (.NET 8) · Modularni monolit · Clean & Hexagonal arhitektura

Sadržaj: opis sistema · arhitektonski dijagram · C4 model ·
arhitektonske odluke · struktura projekta · opis implementacije

1. Opis sistema

TaskFlow je desktop aplikacija za **upravljanje projektima i zadacima**, izrađena u **WPF / .NET 8**. Cilj sistema je da timovima omogući kreiranje projekata, dodelu i praćenje zadataka kroz njihov životni ciklus, automatska obaveštenja o promenama, kao i izveštaje o napretku i produktivnosti.

Sistem je projektovan da bude **arhitektonski uzoran** — implementira modularni monolit sa jasnim granicama između poslovnih oblasti, Clean i Hexagonal arhitekturu, CQRS razdvajanje komandi i upita, te event-driven komunikaciju između modula.

Glavne funkcionalnosti

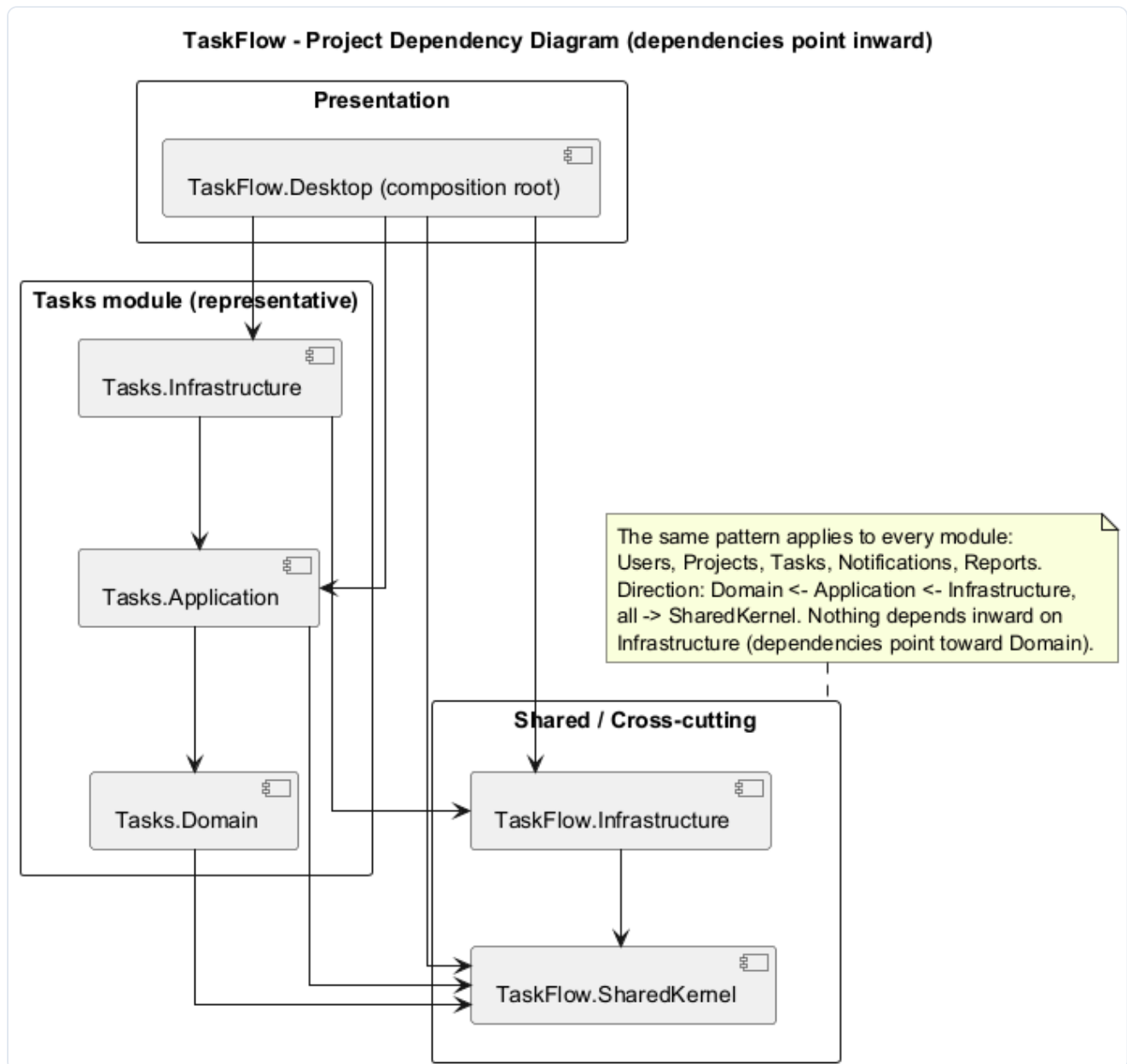
- **Korisnici:** registracija, prijava, dodela rola.
- **Projekti:** kreiranje, izmena, brisanje, upravljanje članstvom, arhiviranje.
- **Zadaci:** CRUD, dodela korisniku, životni ciklus `Todo` → `InProgress` → `Done` , prioriteti, komentari.
- **Obaveštenja:** automatski se kreiraju reagovanjem na događaje (dodela/završetak zadatka).
- **Izveštaji:** statistika projekta i produktivnost korisnika (agregacija/kalkulacija).

Ukupno je implementirano **26 use case-ova** (17 komandi + 9 upita) kroz 5 modula, sa više povezanih entiteta, osnovnim CRUD operacijama, složenom operacijom (workflow + kalkulacija) i domenskim događajima.

Kategorija	Tehnologija
UI	WPF (.NET 8), XAML
Arhitektura	Modularni monolit, Clean Architecture, Hexagonal (Ports & Adapters)
Dependency Injection	Microsoft.Extensions.DependencyInjection
Perzistencija	JSON i SQLite (zamenljivo kroz DI)
Testiranje	xUnit (+ in-memory fake repozitorijumi)
CI/CD	GitHub Actions (build + test)

2. Arhitektonski dijagram (moduli i zavisnosti)

Dijagram prikazuje smer zavisnosti između projekata po Clean Architecture principu. Zavisnosti uvek pokazuju „ka unutra” — ka `Domain` sloju. Prikazan je reprezentativni *Tasks* modul; isti šablon (`Domain` ← `Application` ← `Infrastructure`, svi → `SharedKernel`) važi za sve module. Sloj `TaskFlow.Desktop` je kompozicioni koren koji povezuje `Application` i `Infrastructure` svih modula.

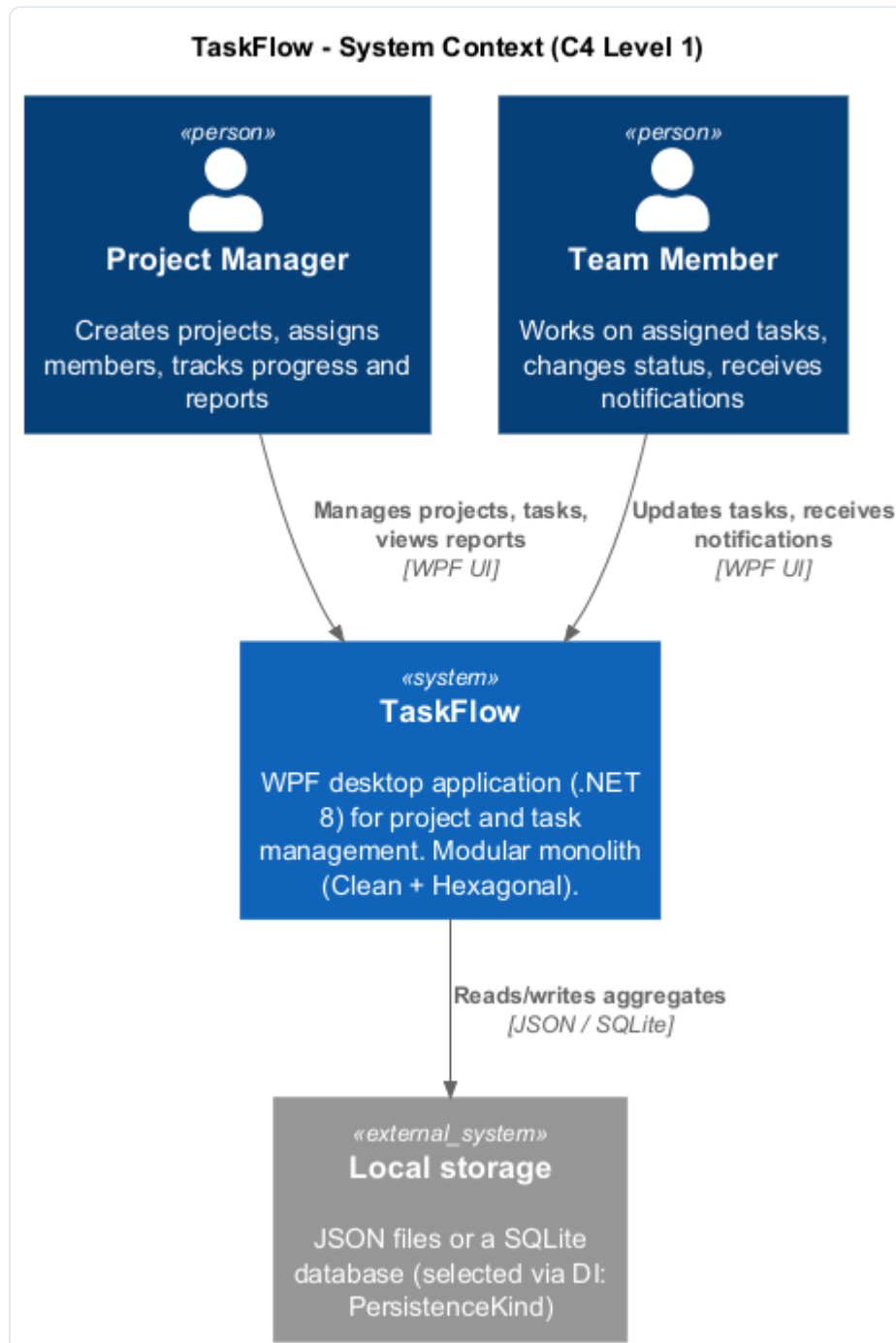


Dijagram zavisnosti projekata (smer ka unutra)

3. C4 dijagram

3.1. Nivo 1 — System Context

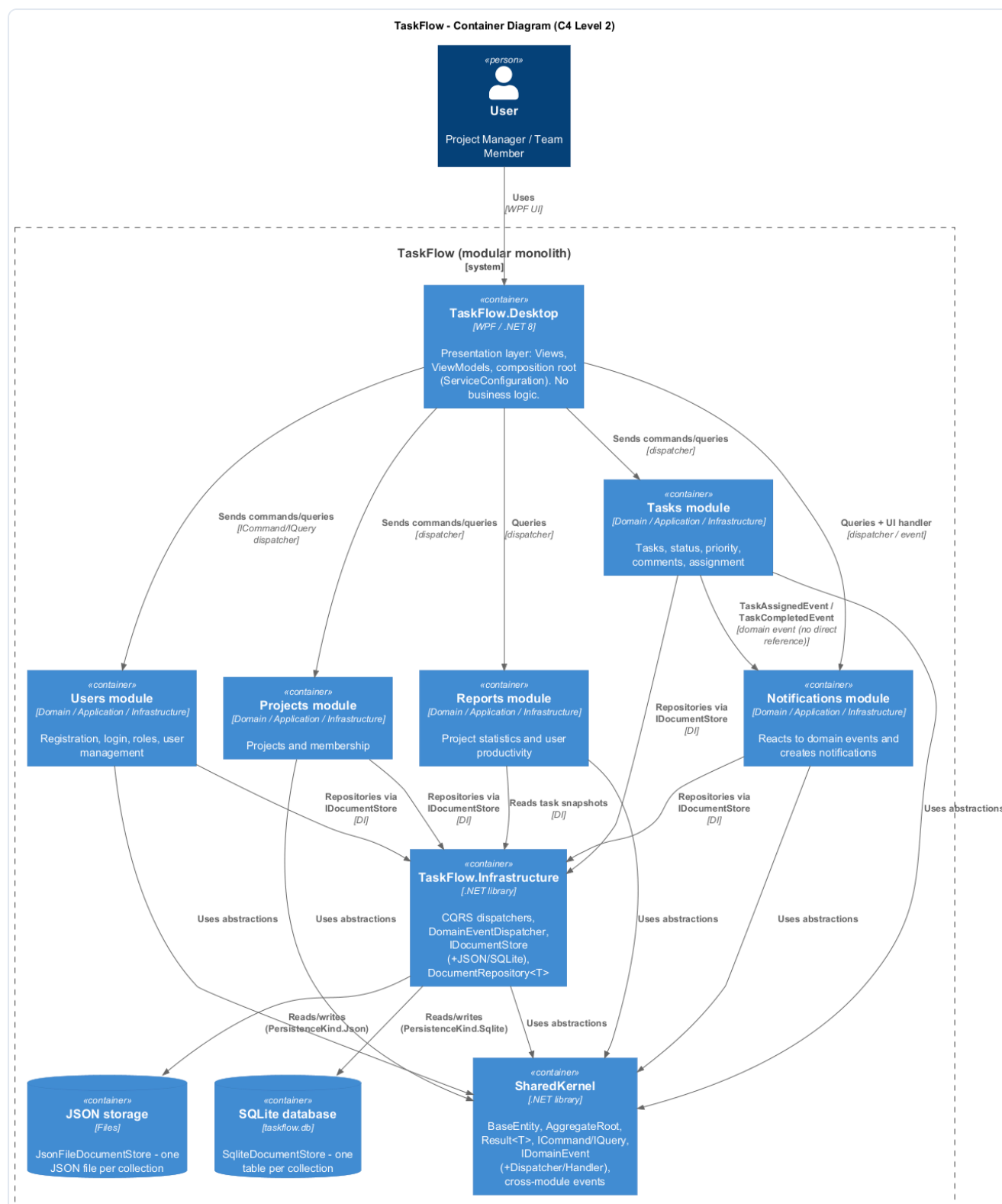
Korisnici (Project Manager, Team Member) koriste TaskFlow desktop sistem, koji čita i piše podatke u lokalno skladište (JSON ili SQLite).



C4 Level 1 — System Context

3.2. Nivo 2 — Container

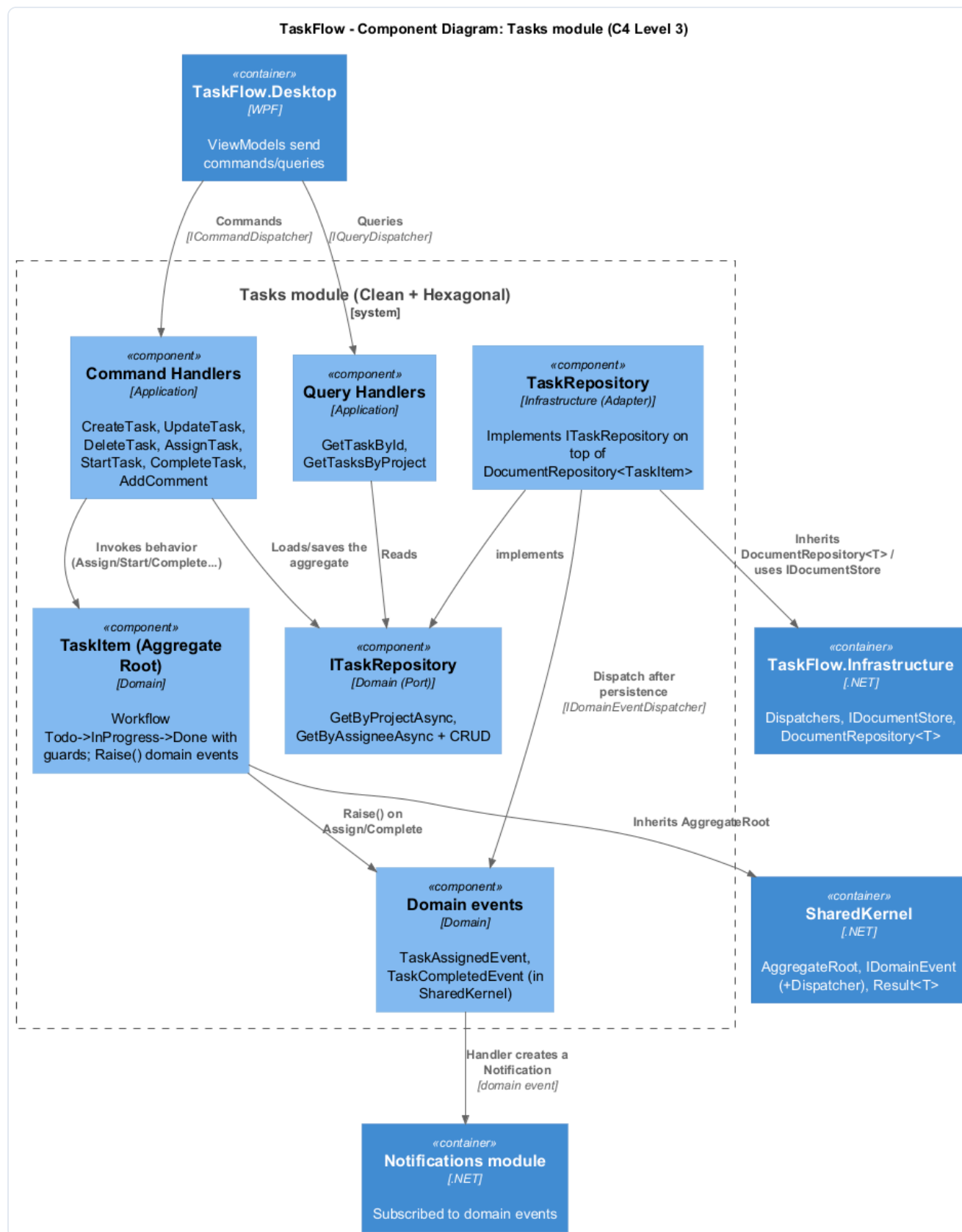
Prikazuje WPF prezentacioni sloj, 5 poslovnih modula, deljene projekte (`SharedKernel` i `TaskFlow.Infrastructure`) i dva skladišta (JSON/SQLite). Vidljiva je i event-driven veza `Tasks` → `Notifications`.



C4 Level 2 — Container Diagram

3.3. Nivo 3 — Component (Tasks modul)

Uvećan *Tasks* modul: Domain (*TaskItem* agregat, port *ITaskRepository* , domenski događaji), Application (handler-i komandi/upita) i Infrastructure (*TaskRepository* adapter), zajedno sa tokom objave događaja posle perzistencije.



C4 Level 3 — Component Diagram (Tasks modul)

4. Opis arhitektonskih odluka

ADR-001 — Modularni monolit

Aplikacija je jedan izvršni proces sa 5 nezavisnih modula (Users, Projects, Tasks, Notifications, Reports). Svaki modul ima svoje slojeve i svoj DI registar; moduli se ne referenciraju direktno. Prednosti: jasne granice, jednostavan deployment, mogućnost kasnijeg izdvajanja modula u zaseban servis.

ADR-002 — Clean + Hexagonal arhitektura

Svaki modul je podeljen na `Domain`, `Application` i `Infrastructure`, sa zavisnostima ka unutra. `Application/Domain` definišu **portove** (interfejs: `I*Repository`, `dispatcher-i`), a `Infrastructure` pruža **adapte**re koji se ubrizgavaju kroz DI. Poslovni kod ne zna koja je konkretna implementacija aktivna, pa je testabilan u izolaciji.

ADR-003 — Perzistencija preko `IDocumentStore` (JSON/SQLite)

Uvedena je apstrakcija `IDocumentStore` sa dve implementacije: `JsonFileDocumentStore` i `SqliteDocumentStore`. Generički `DocumentRepository<T>` radi nad tom apstrakcijom. Izbor implementacije je konfigurabilan kroz DI (enum `PersistenceKind`), bez ikakve izmene poslovne logike. U testovima se koriste in-memory implementacije istih portova.

ADR-004 — Event-driven komunikacija

Agregati objavljuju domenske događaje (npr. `TaskAssignedEvent`, `TaskCompletedEvent`). Posle perzistencije, `DomainEventDispatcher` preko DI pronalazi i poziva sve `IDomainEventHandler<TEvent>`. Tako Notifications modul reaguje na događaje iz Tasks modula **bez direktne reference** na njega, što čuva labavu spregnutost.

5. Struktura projekta (moduli i slojevi)

Svaki poslovni modul prati istu troslojnu strukturu (Domain / Application / Infrastructure). Deljeni projekti i WPF prezentacija su izdvojeni.

```
TaskFlow.sln
├── src/
│   ├── TaskFlow.SharedKernel/      deljene apstrakcije (BaseEntity, AggregateRoot,
│   │                               Result<T>, ICommand/IQuery, IDomainEvent, događaji)
│   ├── TaskFlow.Infrastructure/    adapteri: CQRS/event dispatcher-i, IDocumentStore
│   │                               (JSON/SQLite), DocumentRepository<T>
│   ├── TaskFlow.Desktop/          WPF prezentacija + kompozicioni koren (DI)
│   └── Modules/
│       ├── Users/                 (.Domain / .Application / .Infrastructure)
│       ├── Projects/              (.Domain / .Application / .Infrastructure)
│       ├── Tasks/                 (.Domain / .Application / .Infrastructure)
│       ├── Notifications/         (.Domain / .Application / .Infrastructure)
│       └── Reports/               (.Domain / .Application / .Infrastructure)
├── tests/
└── TaskFlow.Tests/                xUnit + in-memory fake repozitorijumi
```

Uloga slojeva

Sloj	Odgovornost	Zavisi od
Domain	Entiteti/agregati, pravila i invarijante, portovi (interfejsi), domenski događaji	samo SharedKernel
Application	Use case-ovi (komande/upiti i handler-i), DTO-ovi	Domain
Infrastructure	Adapteri: implementacije repozitorijuma, perzistencija	Application, Domain
Presentation	WPF View/ViewModel, DI kompozicioni koren	Application + Infrastructure (svih modula)

Modul	Odgovornost	Use case-ovi
Users	Registracija, login, role	3 komande, 2 upita
Projects	Projekti i članstvo	6 komandi, 2 upita
Tasks	Zadaci, status, prioritet, komentari	7 komandi, 2 upita
Notifications	Obaveštenja iz događaja	1 komanda, 1 upit, 2 event handler-a
Reports	Statistika i produktivnost	2 upita

6. Kratak opis implementacije

CQRS (komande i upiti)

Pisanje i čitanje su razdvojeni. Komande (npr. `CreateTaskCommand`, `AssignTaskCommand`) menjaju stanje preko agregata, a upiti (npr. `GetTasksByProjectQuery`) vraćaju DTO-ove. Dispatcher-i (`ICommandDispatcher`, `IQueryDispatcher`) usmeravaju poziv ka odgovarajućem handler-u razrešenom iz DI kontejnera.

Domenski model i workflow

Agregat `TaskItem` čuva invarijante životnog ciklusa: zadatak se može *startovati* samo iz stanja `Todo`, završen zadatak se ne može ponovo dodeliti, a prelazi su zaštićeni (guard-ovi). Pri dodeli/završetku agregat poziva `Raise(...)` i beleži domenski događaj.

Perzistencija i zamenljivost

Repozitorijumi nasleđuju `DocumentRepository<T>` nad `IDocumentStore`. Izbor skladišta je jedna linija u kompozicionom korenu:

```
// src/TaskFlow.Desktop/ServiceConfiguration.cs
services.AddInfrastructure(PersistenceKind.Json, dataDirectory); // ili PersistenceKind.Sql

// src/TaskFlow.Infrastructure/DependencyInjection.cs
services.AddSingleton<IDocumentStore>(persistence switch
{
    PersistenceKind.Sqlite => new SqliteDocumentStore(Path.Combine(dataDirectory, "taskflow.db"),
    PersistenceKind.Json => new JsonFileDocumentStore(dataDirectory)
});
```

Tok događaja (event-driven)

Posle upisa agregata, `DocumentRepository<T>` objavljuje skupljene događaje preko `IDomainEventDispatcher`. Handler-i u Notifications modulu (npr. `TaskAssignedNotificationHandler`) kreiraju `Notification`, a prezentacioni sloj (`NotificationCreatedUiHandler`) reaktivno osvežava UI.

Testiranje

Domenski sloj se testira jediničnim testovima agregata. Application sloj se testira u izolaciji od infrastrukture korišćenjem **fake/in-memory** implementacija portova (`InMemoryTaskRepository`, `InMemoryNotificationRepository`, `FakePasswordHasher` ...) — bez baze i fajlova.

CI/CD

GitHub Actions workflow (`.github/workflows/ci.yml`) na svaki push/PR ka `main` / `dev` radi `restore` → `build` (Release) → `test` , čime se osigurava da kod uvek prevodi i da svi testovi prolaze.